INCIDENT RESPONSE RUNBOOK

NexaHR Platform | Nexova Technologies Pvt. Ltd.

Version	2.4
Last Updated	January 2025
Owner	Engineering — SRE Team
Classification	INTERNAL USE ONLY
Availability SLA	99.95% uptime
Platform Users	~85,000 end users / 12,000 peak concurrent

NOTE This runbook is used by on-call engineers during production incidents. Read the relevant section quickly and act. If in doubt on severity, escalate up.

1. Severity Classification

SEV	Definition	Example	Response	Resolution	Who Gets Paged
SEV1	Complete outage OR confirmed data breach	NexaHR login down; unauthorized data access detected	5 min	4 hrs	Entire SRE team + Engineering Leads + CEO
SEV2	Major feature unavailable; >20% users affected	Payroll failing; bulk reports not generating	15 min	8 hrs	On-call SRE + Service Owner
SEV3	Degraded performance; <20% users affected	Slow dashboard load for subset of tenants	30 min	24 hrs	On-call SRE only
SEV4	Minor issue; no measurable user impact	UI label misalignment; non-critical log warnings	Next biz day	72 hrs	Ticket only (Jira)

NOTE SLA Baseline: NexaHR targets 99.95% uptime across ~85,000 end users and up to 12,000 peak concurrent users. Uptime deviation is captured in postmortem impact sections.

2. On-Call Rotation

PagerDuty Schedule

- Primary On-Call: Rotates weekly among SRE team members — PagerDuty Schedule: nexahr-sre-primary
- Secondary On-Call: Senior engineer from the affected service team — PagerDuty Schedule: nexahr-sre-secondary

Escalation Path

If primary is unreachable after 15 minutes:

```
Primary On-Call
  No response in 15 min  -->  Secondary On-Call
    No response in 15 min  -->  Usman Baig (VP Operations)
      SEV1 only  -->  Notify CEO via direct call
```

Handoff Process

When transferring incident ownership, post the following in the active #inc-* Slack channel:

```
HANDOFF SUMMARY
_____________________________________
Incident:           [SEV level] — [Short description]
Slack Channel:      #inc-YYYYMMDD-short-description
Duration Active:    [X hrs Y min]
Current Status:     [Investigating / Mitigated / Monitoring]
Last Action Taken:  [What was done]
Next Steps:         [What the incoming engineer should do]
Outstanding Issues: [Open questions]
Handing off to:     [@engineer]
```

Out-of-Hours Escalation Policy

- SEV1 / SEV2: Page immediately regardless of time zone
- SEV3: Page only if unresolved after 2 hrs during business hours; otherwise log and handle next day
- SEV4: No out-of-hours paging — create Jira ticket

3. Step-by-Step Incident Response Process

Step 1 — Detection

Automated Alert: PagerDuty fires from Prometheus/Grafana threshold breach or ELK anomaly detection.
User Report: Submitted via support portal or client email to on-call SRE.

Verify immediately:

1. Check Grafana > Dashboard: NexaHR Platform Overview — service health, error rates, latency P95/P99
2. Check Kibana > Index: nexahr-prod-* — filter level:ERROR for last 15 minutes
3. Check Jaeger — trace search for failed spans on the affected service
4. Check AWS Console > RDS Multi-AZ status (Mumbai + London regions)
5. Confirm scope: isolated tenant vs. platform-wide

Step 2 — Declaration

6. Determine severity using the table in Section 1
7. Create dedicated Slack channel: #inc-YYYYMMDD-short-description

Example:

```
#inc-20250315-payroll-latency
```

8. Post initial alert (see Communication Templates — Section 5)
9. Assign roles: Incident Commander (IC), Tech Lead, Comms Lead

Step 3 — Communication

- Internal Slack update every 30 minutes in the #inc-* channel (template in Section 5)
- External status page (status.nexovatech.com): Update within 10 minutes of SEV1/SEV2 declaration
- Client email for SEV1/SEV2: Sent by Comms Lead within 30 minutes (template in Section 5)

Step 4 — Investigation

```
# Check pod health across all 7 NexaHR microservices
kubectl get pods -n nexahr-prod

# View recent logs for a specific service
kubectl logs -n nexahr-prod -l app=payroll-service --tail=200

# Check Istio service mesh traffic errors
kubectl exec -n istio-system deploy/istiod -- pilot-discovery request GET /debug/syncz

# Kong API Gateway error rate
curl -s http://kong-admin:8001/status | jq '.server'

# Redis cache health (ElastiCache)
redis-cli -h <elasticache-endpoint> ping
redis-cli -h <elasticache-endpoint> info stats | grep rejected_connections

# PostgreSQL active connections
psql -h <rds-endpoint> -U nexahr_admin -c \
  "SELECT count(*), state FROM pg_stat_activity GROUP BY state;"

# Kafka broker lag check
kafka-consumer-groups.sh --bootstrap-server <kafka-broker>:9092 \
  --describe --group nexahr-core
```

Step 5 — Mitigation vs. Resolution

	Mitigation	Resolution
Definition	Stops user impact; system may not be fully healthy	Root cause fixed; system fully restored
Action	Restart pods, redirect traffic, enable fallback	Deploy fix, run smoke tests, confirm metrics normal
Status Page	"Issue identified; fix in progress"	"Resolved"

Step 6 — Postmortem Requirement

- SEV1: Postmortem published within 48 hours of resolution
- SEV2: Postmortem published within 1 week of resolution
- Owner: Incident Commander — use template in Section 6

4. Common Runbooks

Runbook A — Payroll Service High Latency

Symptoms:

- P95 latency >3 s on /api/v1/payroll/*
- PagerDuty alert: payroll-latency-p95-critical

Likely Causes:

10. PostgreSQL slow query on payroll calculation tables (missing index or large dataset)
11. Redis cache miss storm causing DB overload
12. Kafka consumer lag causing event backlog
13. payroll-service pod CPU throttling — check HPA limits
14. Upstream dependency failure (tax-service or compliance-service timeout)

Diagnostic Steps:

```
# 1. Check pod resource usage
kubectl top pods -n nexahr-prod -l app=payroll-service

# 2. Identify slow queries in PostgreSQL
psql -h <rds-endpoint> -U nexahr_admin -c "
  SELECT query, mean_exec_time, calls
  FROM pg_stat_statements
  WHERE query ILIKE '%payroll%'
  ORDER BY mean_exec_time DESC LIMIT 10;"

# 3. Check Redis cache hit ratio
redis-cli -h <elasticache-endpoint> info stats | grep -E 'keyspace_hits|keyspace_misses'

# 4. Kafka lag for payroll consumer
kafka-consumer-groups.sh --bootstrap-server <kafka-broker>:9092 \
  --describe --group payroll-processor | grep -v 0$

# 5. Grafana Panel: 'Payroll Service - Upstream Dependency Latency'
#    Jaeger: search failed spans on payroll-service
```

Mitigation Steps (execute in order):

15. Scale payroll-service pods: kubectl scale deployment payroll-service -n nexahr-prod --replicas=6
16. Flush stale Redis keys: redis-cli -h <endpoint> DEL payroll:cache:*
17. Restart Kafka consumer group if lag >10,000: kubectl rollout restart deployment/payroll-consumer -n nexahr-prod
18. If DB issue confirmed, enable read replica routing via Kong upstream config
19. If latency persists >20 minutes, escalate to SEV2

Runbook B — Authentication Service Outage

Symptoms:

- Login failures platform-wide; 401 errors on all authenticated endpoints
- PagerDuty alert: auth-service-down

Scenario 1 — Redis Cache Failure:

```
# Check ElastiCache status
redis-cli -h <elasticache-endpoint> ping

# If no response, check AWS Console -> ElastiCache -> nexahr-session-cache

# Enable fallback to DB-backed session auth
kubectl set env deployment/auth-service -n nexahr-prod SESSION_BACKEND=database
kubectl rollout restart deployment/auth-service -n nexahr-prod
```

Scenario 2 — JWT Signing Key Rotation:

NOTE Coordinate with Ayesha Naqvi (a.naqvi@nexova.io) before key rotation for audit trail requirements.

```
# Verify current key status
kubectl get secret jwt-signing-key -n nexahr-prod -o jsonpath='{.data.key}' \
  | base64 -d | openssl rsa -noout -text

# Rotate signing key
kubectl create secret generic jwt-signing-key-new \
  --from-literal=key="$(openssl genrsa 2048)" -n nexahr-prod

kubectl patch deployment auth-service -n nexahr-prod -p \
  '{"spec":{"template":{"spec":{"volumes":[
    {"name":"jwt-key","secret":{"secretName":"jwt-signing-key-new"}}
  ]}}}}'

# Allow 5-min grace period for existing tokens before removing old key
```

Emergency Fallback — Maintenance Mode via Kong:

```
curl -X POST http://kong-admin:8001/plugins \
  -d "name=request-termination" \
  -d "config.status_code=503" \
  -d "config.message=Auth maintenance in progress"
```

Runbook C — Database Connection Pool Exhaustion

Detection:

- Grafana Panel: PostgreSQL — Active Connections (threshold: >450 of 500 max)
- Grafana Panel: NexaHR Services — DB Connection Wait Time (threshold: >500 ms)
- PagerDuty alert: pg-connection-pool-critical

Immediate Mitigation:

```
# 1. Identify top connection consumers
psql -h <rds-endpoint> -U nexahr_admin -c "
  SELECT application_name, count(*), state
  FROM pg_stat_activity
  GROUP BY application_name, state
  ORDER BY count DESC;"

# 2. Terminate idle connections older than 10 minutes
psql -h <rds-endpoint> -U nexahr_admin -c "
  SELECT pg_terminate_backend(pid)
  FROM pg_stat_activity
  WHERE state = 'idle'
  AND query_start < NOW() - INTERVAL '10 minutes';"

# 3. Reduce pool size temporarily on top consumer
kubectl set env deployment/payroll-service -n nexahr-prod DB_POOL_MAX=10
```

Root Cause Investigation:

20. Review PgBouncer logs (if enabled) for pool config drift
21. Check recent deployments in ArgoCD for connection config changes

22. Review Kafka consumer lag — high lag causes retry storms that exhaust connections
23. Check GitHub Actions recent merges for misconfigured DB pool settings

5. Communication Templates

Initial Internal Slack Alert

```
INCIDENT DECLARED -- [SEV1 / SEV2 / SEV3]
==========================================
Channel:      #inc-YYYYMMDD-short-description
Summary:      [1-2 sentence description of what is broken]
Impact:       [Estimated % of users / which features affected]
IC:           @[incident-commander]
Tech Lead:    @[engineer]
Comms Lead:   @[person]
Next update:  30 minutes
```

30-Minute Progress Update (Slack)

```
UPDATE -- [HH:MM UTC]
==============================
Status:       [Investigating / Mitigated / Monitoring]
What we know: [Current findings]
Actions:      [What we are doing right now]
Next update:  30 minutes / ETA for resolution: [time]
```

Status Page Updates (status.nexovatech.com)

Investigating

We are aware of an issue affecting [feature/service] and are actively investigating. We will provide an update within 30 minutes.

Identified

We have identified the root cause of the issue affecting [feature/service]. Our engineering team is deploying a fix. Estimated resolution: [time].

Resolved

This incident has been resolved. [Feature/service] is fully operational as of [HH:MM UTC]. We apologize for the disruption and will publish a postmortem within [48 hours / 1 week].

Client Email — SEV1 (from CTO)

```
Subject: NexaHR Service Disruption — [Date] — Action Update

Dear [Client Name],

We are writing to inform you that NexaHR experienced a service disruption
beginning at [HH:MM UTC] on [Date]. The issue affected [describe impact,
e.g., login access / payroll processing] for your organization.

Our engineering team identified and resolved the issue at [HH:MM UTC].
Total duration: [X hours Y minutes].

We take our 99.95% uptime commitment seriously and sincerely apologize
```

```
for the impact on your operations. A full incident report will be shared
within 48 hours.

If you have questions, please contact your dedicated account manager
or reach us at support@nexovatech.com.

Regards,
[CTO Name]
Chief Technology Officer
Nexova Technologies Pvt. Ltd.
```

6. Postmortem Template

NOTE SEV1: Publish within 48 hours of resolution. SEV2: Publish within 1 week. Owner: Incident Commander. Reviewer: SRE Lead + Usman Baig (VP Operations) for SEV1.

Incident Summary

Incident ID	INC-YYYYMMDD-XXX
Severity	SEV[1/2]
Date	[Date of incident]
Duration	[X hrs Y min]
Author (IC)	[Name]
Reviewed by	[SRE Lead / Usman Baig for SEV1]
Published	[Date]

Provide a 2-3 sentence description of what happened, which NexaHR services were affected, and the user/business impact.

Timeline (5-minute granularity, UTC)

```
HH:MM  Alert fired / User report received
HH:MM  On-call SRE acknowledged
HH:MM  Incident declared; Slack channel created
HH:MM  Initial investigation begun
HH:MM  Root cause hypothesis formed
HH:MM  Mitigation applied
HH:MM  User impact ended (mitigation effective)
HH:MM  Full resolution confirmed
HH:MM  Incident closed
```

Root Cause

[Technical description of the exact failure — be precise.]

Contributing Factors:

- [Factor 1]
- [Factor 2]
- [Factor 3]

Impact

Users affected	[Number / % of ~85,000]
Duration	[X hrs Y min]
Revenue impact (est.)	[USD amount if calculable — ref Q1 actuals USD 2.34M if needed]
SLA impact	[Uptime % for the period vs. 99.95% target]
Client notifications	[Yes/No — which clients]

Action Items

#	Action	Owner	Due Date	Status
1	[Corrective action]	[@engineer]	[Date]	Open
2	[Preventive measure]	[@engineer]	[Date]	Open

What Went Well

- [e.g., PagerDuty fired within expected threshold]
- [e.g., Runbook A reduced MTTR significantly]

What Did Not Go Well

- [e.g., Slack channel took 12 min to create — delayed comms]
- [e.g., Jaeger traces unavailable due to sampling config]

Contacts: Legal/Security (data breach SEV1): Ayesha Naqvi — a.naqvi@nexova.io | Finance impact: Bilal Chaudhry — b.chaudhry@nexova.io | SRE queries: #sre-internal (Slack)